SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5115

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I _____meghanadh A (192372281)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *meghanadh*

Annexure A
Debugging & Optimisation Task

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B. Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

2A)

Performance Analysis of Block Cipher Modes of Operation

A. Debug Incorrect Padding and IV Handling Causing Decryption Failures

1. Incorrect Padding

- AES requires the plaintext length to be a multiple of 16 bytes.
- If padding is not added correctly during encryption or removed correctly during decryption, decryption fails or produces incorrect output.
- Use PKCS#7 padding to ensure compatibility.

Correct Padding Example (Python):

```
from Crypto.Util.Padding import pad, unpad

plaintext = b"Hello World"

# Encryption

padded_data = pad(plaintext, 16)

# Decryption

original_data = unpad(padded_data, 16)
```

2. Incorrect IV (Initialization Vector) Handling

- **ECB mode** does not require an IV.
- **CBC, CFB, and OFB modes** require a **16-byte IV**.
- The same IV used during encryption must also be used during decryption.
- Using a different IV during decryption results in incorrect plaintext.

Correct IV Handling Example (Python):

```
from Crypto.Cipher import AES

from Crypto.Random import get_random_bytes

key = get_random_bytes(16)

iv = get_random_bytes(16)

# Encryption

cipher = AES.new(key, AES.MODE_CBC, iv)

# Decryption (same IV)

cipher = AES.new(key, AES.MODE_CBC, iv)
```

B. Refactor the Code to Minimize Redundant Computations and Memory Usage

Problems Identified

- Creating multiple AES cipher objects unnecessarily.
- Storing duplicate copies of plaintext and ciphertext.
- Applying padding repeatedly.
- Reading the entire file into memory.

Optimized Solution

- Create cipher objects only when needed.
- Apply padding only once.
- Process data block by block.
- Reuse memory buffers wherever possible.
- Avoid unnecessary data copying.

Optimized Python Code :

```
from Crypto.Cipher import AES

from Crypto.Util.Padding import pad, unpad

from Crypto.Random import get_random_bytes

key = get_random_bytes(16)

iv = get_random_bytes(16)

def encrypt(data):

    cipher = AES.new(key, AES.MODE_CBC, iv)
```

```
    return cipher.encrypt(pad(data, AES.block_size))

def decrypt(ciphertext):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return unpad(cipher.decrypt(ciphertext), AES.block_size)
```

C. Compare and Optimize the Implementation

Mode	IV Required	Security	Speed	Memory Usage	Real-Time Communication
ECB	No	Low	Very Fast	Low	Not Recommended
CBC	Yes	High	Moderate	Medium	Suitable
CFB	Yes	High	Moderate	Medium	Better
OFB	Yes	High	Fast	Low	Best

Optimization Techniques

- Use hardware-accelerated AES whenever available.
- Reuse the same IV for decryption.
- Process large files in small chunks.
- Eliminate redundant memory allocations.
- Avoid repeated padding operations.
- Reuse buffers to reduce memory usage.

Conclusion

- **PKCS#7 padding** prevents decryption failures caused by incorrect plaintext length.
- Correct IV management ensures successful decryption in **CBC, CFB, and OFB** modes.
- Refactoring reduces memory usage and improves execution speed.
- **OFB mode** is the most efficient for real-time secure communication because it provides low latency, low memory usage, and high security. **CBC** and **CFB** are also secure options, whereas **ECB** is not recommended due to its poor security despite its high speed.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

3A)
Debugging RSA Cryptosystem Implementation
Aim : To identify and fix errors in an RSA cryptosystem implementation, optimize its performance using efficient algorithms, and evaluate the trade-offs between security and computational efficiency.

A. Debugging the RSA Cryptosystem

1. Issues in Prime Generation

Problem

- Small prime numbers make RSA vulnerable to brute-force attacks.
- Non-prime numbers may be mistakenly selected.
- Predictable random number generators reduce security.

Solution

- Use probabilistic primality tests such as **Miller–Rabin**.
- Generate large random primes (2048-bit or higher).
- Use cryptographically secure random number generators.

2. Issues in Key Generation

Problem

- Choosing a public exponent (**e**) that is not coprime with φ(n).
- Incorrect calculation of the private key (**d**).
- Using weak key sizes (512-bit or 1024-bit).

Solution

- Select

e = 65537

(standard secure public exponent).

- Verify

  gcd(e, φ(n)) = 1

- Compute

  $d = e^{-1} \bmod \varphi(n)$

  using the Extended Euclidean Algorithm.

- Use at least **2048-bit** keys.

3. Issues in Modular Exponentiation

Problem

- Direct computation of

  M^e mod n

  causes overflow and is extremely slow.

- Large messages exceed the modulus size.

Solution

- Use **Fast Modular Exponentiation (Square-and-Multiply)**.
- Split large messages into blocks smaller than the modulus.
- Apply secure padding schemes such as **OAEP**.

B. Algorithm Optimization

1. Fast Modular Exponentiation

Instead of computing

C = M^e mod n

directly,

use the **Square-and-Multiply Algorithm**.

Advantages

- Reduces time complexity from **O(e)** to **O(log e)**.
- Handles very large exponents efficiently.
- Prevents integer overflow.

2. Chinese Remainder Theorem (CRT)

During decryption,

instead of computing

M = C^d mod n

perform:

- Compute modulo **p**
- Compute modulo **q**
- Combine the two results using CRT.

Advantages

- Nearly **4× faster** decryption.
- Widely used in practical RSA implementations.
- Reduces computational workload.

3. Additional Optimizations

- Use efficient big-integer arithmetic libraries.
- Cache repeated computations where appropriate.
- Use secure padding (OAEP).
- Encrypt only small data and use **hybrid encryption** (RSA + AES) for large files.

C. Trade-offs Between Key Size, Security, and Performance

Key Size	Security Level	Encryption Speed	Decryption Speed	Recommended Use
1024-bit	Low (deprecated)	Very Fast	Fast	Not recommended
2048-bit	High	Fast	Moderate	Standard application
3072-bit	Very High	Moderate	Slower	Government and enterprise
4096-bit	Extremely High	Slow	Very Slow	Highly sensitive data

Performance Analysis

Small Keys (1024-bit)

Advantages

- Faster encryption and decryption.
- Lower memory usage.

Disadvantages

- Vulnerable to modern attacks.
- No longer considered secure.

Medium Keys (2048-bit)

Advantages

- Good balance between security and performance.
- Recommended by most security standards.

Disadvantages

- Slightly slower than 1024-bit RSA.

Large Keys (4096-bit)

Advantages

- Strong protection against factorization attacks.
- Suitable for long-term security.

Disadvantages

- High computational cost.
- Increased memory and processing requirements.

Optimization Summary

Component	Issue	Optimization	Benefit
Prime Generation	Weak or non-prime values	Miller–Rabin Test	Stronger security
Key Generation	Invalid public/private keys	Extended Euclidean Algorithm	Correct key generation
Modular Exponentiation	Slow computation	Square-and-Multiply	Faster encryption/decryption
Decryption	High computation time	Chinese Remainder Theorem	Up to 4× faster
Large Messages	Encryption failure	Block encryption + OAEP	Reliable encryption

Conclusion

The RSA implementation can fail due to weak prime generation, incorrect key generation, inefficient modular exponentiation, and attempting to encrypt messages larger than the modulus. Using secure prime generation (Miller–Rabin), proper key generation with the Extended Euclidean Algorithm, fast modular exponentiation, and the Chinese Remainder Theorem significantly improves both correctness and performance. A **2048-bit RSA key** provides the best balance between security and computational efficiency for most real-world applications.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation